

Sprint Retrospective			
What went well	What went poorly	What ideas do you have	How should we take action
<i>This section highlights the successes and positive outcomes from the sprint. It helps the team recognize achievements and identify practices that should be continued.</i>	<i>This section identifies the challenges, roadblocks, or failures encountered during the sprint. It helps pinpoint areas that need improvement or change.</i>	<i>This section is for brainstorming new approaches, tools, or strategies to enhance the team's efficiency, productivity, or project outcomes.</i>	<i>This section outlines specific steps or solutions to add and implement the ideas discussed, ensuring continuous improvement in future sprints.</i>
Example : All tasks were completed on time. Team communication was seamless.	Requirements changed mid-sprint.	Plan for a buffer to handle scope changes.	Set stricter deadlines for finalizing requirements.
Flask backend and MCP server architecture were completed on time. All 11 tool endpoints (nmap, gobuster, dirb, nikto, sqlmap, metasploit, hydra, john, wpscan, enum4linux + health + execute_command) are registered and functional.	Reconnaissance items (Gobuster & Dirb, Result Parsing) remain In Progress — wordlist path is not validated and tool output is returned as raw strings with no structured parsing for LLM consumption.	Add a response parser layer in api_server.py that converts raw stdout from each tool into a structured JSON schema (e.g. {open_ports: [], findings: []}) before returning to the MCP caller.	Create a parse_nmap_output(), parse_gobuster_output() module. Integrate parsers into each Flask route before jsonify(result). Target: Sprint 2.
CommandExecutor implements a reliable SIGTERM → SIGKILL (5s grace) timeout mechanism. Partial output captured before timeout is preserved and returned — LLM receives useful data even on long-running scans.	COMMAND_TIMEOUT is hardcoded at 180s in api_server.py with no per-endpoint or per-request override capability. The MCP client uses a separate DEFAULT_REQUEST_TIMEOUT=300s, creating a mismatch where the HTTP connection can outlive the subprocess.	Expose timeout as an optional parameter in each Flask endpoint and pass it into CommandExecutor. Allow MCP tools to override timeout per invocation via an optional timeout argument.	Add optional timeout param to each POST endpoint (params.get('timeout', COMMAND_TIMEOUT)). Update CommandExecutor.__init__ to accept it. Update all @ functions accordingly. Target: Sprint 2.
Metasploit integration uses an .rc resource script approach (/tmp/mcp_msfr_resource.rc) enabling non-interactive execution. Temp file cleanup (os.remove) is implemented after execution.	No API authentication layer exists — all Flask endpoints accept unauthenticated requests. The server is protected only by the default 127.0.0.1 binding; if --ip is changed to 0.0.0.0, the entire attack toolkit becomes publicly accessible with no token check.	Implement a simple API key middleware using Flask's @app.before_request hook. Read a secret token from an environment variable (API_TOKEN) and reject requests missing the X-API-Key header with HTTP 401.	Add before_request auth middleware to api_server.py (API_TOKEN via environment variable). Update KaliToolsClient.safe_post/safe_get to include the token. Document setup in README. Target: Sprint 2 (Must)
KaliToolsClient.safe_get and safe_post use consistent try/except error handling, returning {error: ..., success: False} on failure. All tool errors are logged with logger.error() including tracebacks, making debugging straightforward.	No audit logging is implemented — tool invocations are only written to stdout via standard Python logging. There is no structured record of operator identity, target, command executed, or output summary, which is required for legal defensibility in authorised engagements.	Integrate a structured audit logger that writes JSON-formatted log entries to a rotating file (e.g. /var/log/kalimind/audit.jsonl) on every tool invocation using Python's RotatingFileHandler.	Create an audit_logger.py module with a log_invocation(command, output_summary) function. Call it from each tool after execute_command returns. Target: Sprint 3 (Should)
The /health endpoint checks availability of all essential tools (nmap, gobuster, dirb, nikto) and returns a structured JSON with tools_status dict and all_essential_tools_available flag. MCP client reads health on startup and warns if tools are missing.	Report generation (/mcp/capabilities, /mcp/tools/<tool_name>) endpoints are stub functions with only pass — completely unimplemented. Automated Report Generation (backlog item 16) is blocked with no output format, template, or FR-10 implementation started.	Design a lightweight report schema (JSON → Markdown → HTML) that aggregates all tool outputs from a pentest session. The LLM agent can call a generate_report MCP tool that compiles findings into a structured vulnerability assessment.	Implement /api/report endpoint in api_server.py. Add generate_report MCP tool to mcp_client.py. Define report with sections: target, recon_findings, vulnerabilities, recommendations. Target: Sprint 3.
WPScan integration is marked Done and follows the same clean endpoint pattern as other tools. Hydra uses -t 4 (limited threads) to avoid overwhelming targets — shows awareness of responsible testing constraints.	execute_command tool exposes arbitrary shell command execution on the Kali server with no input sanitisation or allowlist. shell=True in subprocess.Popen combined with unsanitised input creates a command injection risk if the API is ever exposed beyond localhost.	Replace shell=True with shell=False and pass commands as argument lists using shlex.split(). Add an allowlist or regex validation layer for the generic execute_command endpoint to restrict dangerous characters.	Refactor CommandExecutor.execute() to use shlex.split(self.command) and shell=False. Add input validation to the /api/command route. Document risk in security handbook. Target: Sprint 2 (Security).